

Veille Technologique

Microcontrôleurs & Arduino

Groupe

Makhtar WADE

Mouhamed DIOP

Bintou DIAME

Quel est le rôle d'une carte Arduino ?

C'est un petit ordinateur simplifié (un microcontrôleur) sur une carte électronique. Son rôle est de faire le lien entre le monde physique et le code :

- Elle **capte** des informations (température, lumière, présence) grâce à des capteurs connectés.
- Elle **décide** quoi faire en exécutant le programme qu'on a écrit.
- Elle **agit** sur l'environnement en envoyant des signaux à des actionneurs (LEDs, moteurs, écrans).

C'est le cerveau de notre projet de prototypage.



Bouton de réinitialisation

Terre / masse

Entrées / sorties numériques

Port USB

Témoin d'alimentation

Schéma complet des broches et composants de l'Arduino Uno

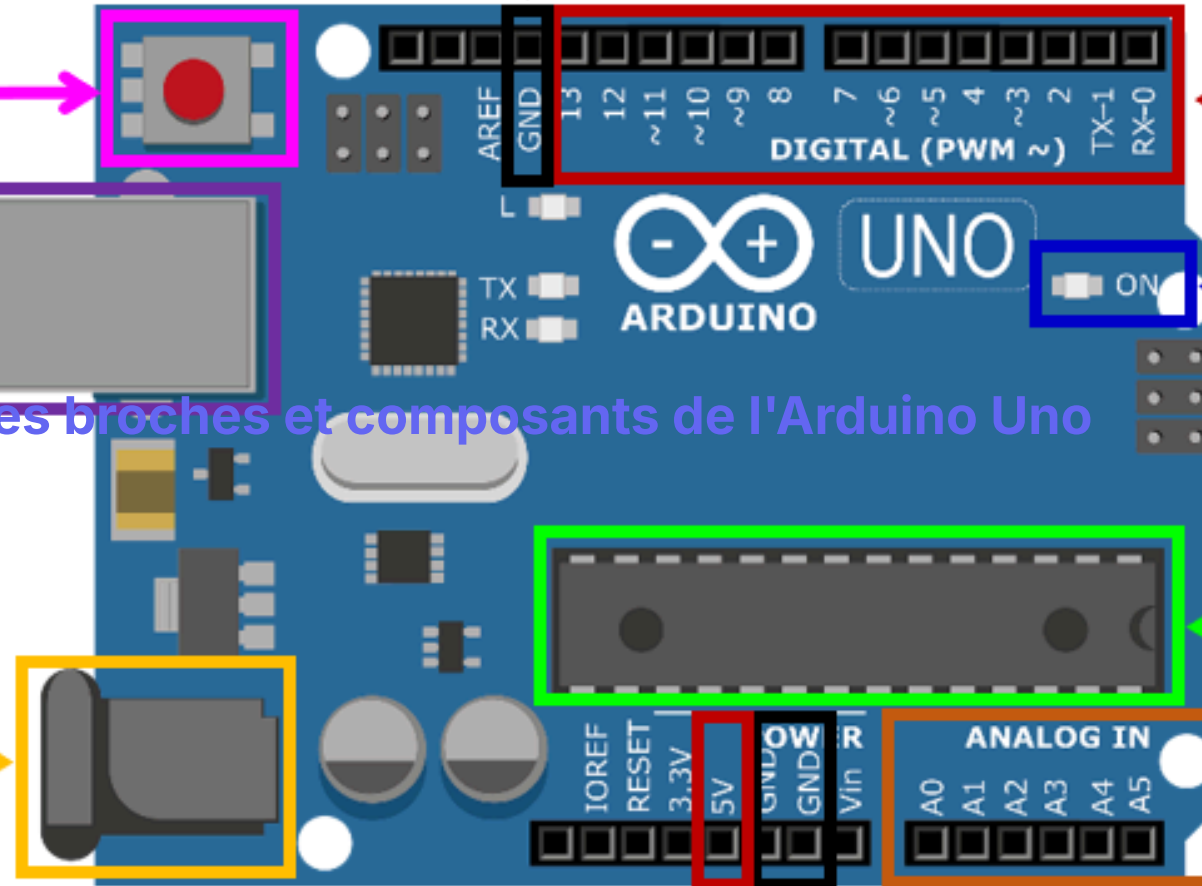
Microcontrôleur

Prise d'alimentation

Entrées analogiques

Alimentation 5 V

Terre / masse



Quelle est la différence entre Arduino Uno, Arduino Mega et Arduino Nano ?

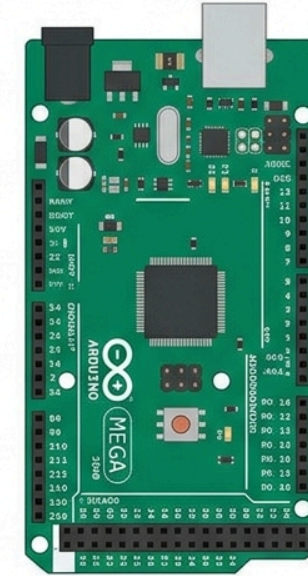
Ces trois cartes partagent le même principe mais s'adaptent à la taille et aux besoins du projet :

- **Arduino Uno** : Le modèle standard. Idéal pour débuter, robuste et facile à câbler.
- **Arduino Mega** : Version XXL. Elle offre beaucoup plus de pins (54) et de mémoire. On l'utilise pour des gros projets (robots complexes, imprimantes 3D).
- **Arduino Nano** : Version miniature. Elle fait presque la même chose que la Uno mais tient sur une breadboard pour les projets compacts.



Arduino Uno

Processor	ATmega328P
Clock speed	16 MHz
Digital pins	14 (6 PWM)
Analog pins	6
RAM	2 KB
Flash memory	32 KB



Arduino Mega 2560

Processor	ATmega2560
Clock speed	16 MHz
Digital pins	54 (15 PWM)
Analog pins	16
RAM	8 KB
Flash memory	256 KB



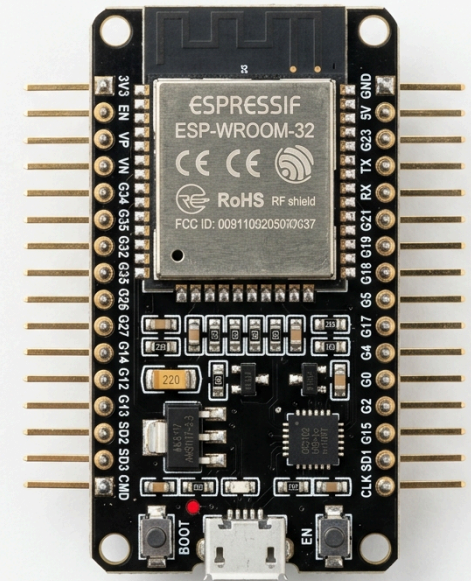
Arduino Nano

Processor	ATmega328P
Clock speed	16 MHz
Digital pins	14 (6 PWM)
Analog pins	8
RAM	2 KB
Flash memory	32 KB

Pourquoi l'ESP32 est-il souvent utilisé pour les projets IoT ?

L'ESP32 s'est imposé dans l'Internet des Objets (IoT) car il résout les manques de l'Arduino Uno :

- **Wi-Fi et Bluetooth intégrés** : Plus besoin de rajouter des modules compliqués et chers.
- **Puissance de calcul** : Il tourne à 240 MHz (double-cœur) contre seulement 16 MHz pour la Uno.
- **Mémoire confortable** : 520 Ko de RAM (contre 2 Ko), indispensable pour sécuriser les données et gérer le réseau.
- **Économie d'énergie** : Des modes de veille très poussés pour fonctionner longtemps sur batterie.



Quelles sont les principales entrées et sorties disponibles sur une carte Arduino ?

Elles se divisent en plusieurs groupes physiques sur la carte :

- **Entrées/Sorties Numériques (D0-D13)** : Pour le "tout ou rien" (allumer une LED, lire un bouton poussoir).
- **Sorties PWM (indiquées par un ~)** : Pour simuler une variation (vitesse d'un moteur, intensité d'une LED).
- **Entrées Analogiques (A0-A5)** : Pour mesurer une tension variable (capteur de lumière, de température).
- **Broches d'alimentation (5V, 3.3V, GND)** : Pour fournir l'énergie électrique aux capteurs et modules.

Quelle est la différence entre une entrée analogique et une entrée numérique ?

La différence réside dans la nature du signal que la carte reçoit :

- **Entrée numérique** : Ne comprend que deux états. C'est 0 (0V) ou 1 (5V). On l'utilise par exemple pour savoir si un bouton est appuyé ou non.
- **Entrée analogique** : Mesure une tension qui varie de façon continue (entre 0V et 5V). La carte convertit cette mesure en un nombre de **0 à 1023**. On l'utilise pour quantifier précisément une valeur (comme la luminosité ambiante avec une LDR).

À quoi servent les fonctions `setup()` et `loop()` dans Arduino ?

Chaque programme Arduino s'organise autour de ces deux fonctions obligatoires :

- `setup()` : Elle s'exécute **une seule fois** au démarrage. On l'utilise pour configurer le système (dire quelles broches sont des entrées ou des sorties, démarrer la liaison avec l'ordinateur).
- `loop()` : Elle tourne **en boucle infinie** dès que `setup()` est finie. C'est ici qu'on met la logique du projet (lire le capteur, calculer, puis allumer le moteur ou l'alarme). Elle se répète des milliers de fois par seconde.

Comment un Arduino communique-t-il avec un ordinateur ?

La liaison se fait via le câble USB, mais de façon indirecte :

1. Le microcontrôleur principal de la carte parle un langage simple appelé **UART** (liaison série).
2. Une petite puce convertisseuse présente sur la carte (comme l'ATmega16U2 ou la CH340) traduit ce signal en **USB** pour que l'ordinateur puisse le lire.
3. Côté code, on utilise la commande `Serial.begin(9600)` pour ouvrir cette ligne de communication.

Qu'est-ce que le moniteur série ?

C'est une fenêtre de l'IDE Arduino qui sert de terminal de discussion entre la carte et votre ordinateur.

- **À quoi ça sert ?** Principalement à déboguer. On peut lui demander d'afficher en direct ce que mesurent les capteurs grâce à `Serial.println(valeur)`.
- **Pourquoi c'est utile ?** Sans écran branché sur la carte, c'est notre seul moyen de voir ce qu'il se passe à l'intérieur du microcontrôleur pendant que le programme tourne.

Quel est le rôle d'un module Wi-Fi ESP8266 ?

Ce module sert de passerelle réseau. Il permet de connecter un Arduino classique (qui n'a pas d'antenne) à internet :

- Il communique avec l'Arduino en liaison série (avec des commandes texte dites "commandes AT").
- Il permet d'envoyer les données de nos capteurs vers un serveur en ligne, ou de piloter l'Arduino à distance depuis une page web.
- *Note* : Il possède aussi son propre processeur et peut parfois être programmé tout seul, sans carte Arduino.

Comment connecter un capteur à une carte Arduino ?

La connexion typique se fait presque toujours avec 3 fils :

1. **L'alimentation (VCC)** : reliée au 5V ou 3.3V de l'Arduino.
2. **La masse (GND)** : branchée sur le GND de la carte pour fermer le circuit électrique.
3. **Le signal (SIG / OUT)** : relié à une pin de l'Arduino.
 - Si le signal est binaire (ex: détecteur de présence) → pin numérique.
 - Si le signal varie continuellement (ex: température analogique) → pin analogique.

Comment commander un actionneur avec Arduino ?

Tout dépend de la puissance électrique dont l'actionneur a besoin :

- **Faible puissance** (LED, petit buzzer) : On le branche directement sur une sortie numérique de l'Arduino en ajoutant une résistance pour limiter le courant.
- **Forte puissance** (moteur, lampe 220V) : L'Arduino ne peut pas fournir assez de courant (limité à 40mA par pin). On utilise alors un composant intermédiaire (transistor, relais ou pont en H) qui sert d'interrupteur commandé. L'énergie provient d'une alimentation externe.

Qu'est-ce qu'une bibliothèque Arduino ?

C'est un ensemble de code écrit par la communauté pour simplifier l'utilisation de composants complexes.

- **L'avantage** : Vous n'avez pas besoin de programmer chaque étape technique. La bibliothèque traduit vos demandes simples en instructions complexes pour la carte.
- **Exemple** : Au lieu de coder manuellement le protocole de communication d'un capteur, vous importez la bibliothèque associée (`#include <Nom.h>`) et utilisez une commande toute faite comme `capteur.read()`.

Quel est le rôle des broches VCC, GND, SIG, AO et DO sur un module électronique ?

Ces abréviations imprimées sur nos modules ont chacune une utilité précise :

- **VCC** : L'alimentation positive (+5V ou +3.3V) venant de la carte Arduino.
- **GND** : La masse (0V), indispensable pour fermer le circuit électrique.
- **SIG** : La broche qui transporte le signal de données mesuré par le capteur.
- **AO** : La sortie analogique (Analog Output) qui donne une tension variable.
- **DO** : La sortie numérique (Digital Output) qui passe à 0 ou 1 si un seuil réglable est dépassé.

Comment identifier les broches d'un capteur lorsque la documentation n'est pas disponible ?

Si vous n'avez pas de mode d'emploi, voici comment procéder :

1. **Regardez la sérigraphie** : Inspectez les deux faces du circuit imprimé, les labels (VCC, GND, OUT, +, -) sont souvent écrits en tout petit.
2. **Trouvez la puce principale** : Tapez sa référence sur internet (ex: "LM393 datasheet") pour trouver le schéma.
3. **Utilisez un multimètre** : En mode continuité, cherchez quelle broche est reliée au grand plan de masse en cuivre (c'est le GND).
4. **Repérez les composants clés** : La présence d'un petit potentiomètre bleu indique souvent une broche DO (Digital Output).

Quelles sont les limites d'une carte Arduino pour une application complexe ?

La Uno montre rapidement ses limites si le projet grandit :

- **Manque de mémoire** : Avec seulement 2 Ko de RAM, on ne peut pas traiter beaucoup de données ou de textes longs.
- **Pas de multitâche** : Elle ne fait qu'une seule chose à la fois. La fonction `delay()` bloque tout le programme.
- **Pas de connectivité** : Aucun circuit Wi-Fi ou Bluetooth n'est intégré d'origine.
- **Manque de puissance** : Son processeur à 16 MHz est trop lent pour du traitement d'images ou du son.

Dans quels cas faut-il préférer ESP32 à Arduino Uno ?

On choisit l'ESP32 dès que les limites de la Uno bloquent le projet :

- **Besoin de connectivité** : Pour envoyer des mesures de température sur un serveur web ou piloter un relais via Wi-Fi ou Bluetooth.
- **Traitement lourd** : Pour faire du calcul mathématique plus complexe ou chiffrer les données de communication.
- **Besoin de rapidité** : Pour gérer des écrans graphiques complexes ou des capteurs rapides.
- **Autonomie sur batterie** : Grâce aux modes sommeil de l'ESP32, la carte consomme presque rien quand elle n'est pas active.

Comment choisir le bon port série dans Arduino IDE ?

Si vous hésitez dans la liste des ports disponibles :

1. Ouvrez le menu **Outils > Port** et regardez les choix.
2. Débranchez votre carte de l'ordinateur.
3. Rouvrez le menu : le port qui a disparu est celui de votre carte.
4. Rebranchez-la et sélectionnez ce port.
 - *Windows* : Il s'appelle généralement `COM3`, `COM4`, etc.
 - *Linux* : Il s'appelle souvent `/dev/ttyACM0` ou `/dev/ttyUSB0`.

Comment choisir le bon modèle de carte dans Arduino IDE ?

Il faut obligatoirement configurer l'IDE pour la bonne puce :

1. Allez dans **Outils > Type de carte**.
2. Sélectionnez la famille correspondante (par exemple **Arduino AVR Boards** pour les cartes classiques).
3. Cliquez sur le nom exact de votre carte (ex: **Arduino Uno**, **Arduino Nano**).
4. *Attention* : Pour l'ESP32, il faut d'abord ajouter son lien d'installation dans les Préférences de l'IDE pour qu'il apparaisse dans la liste.

Quelle est la différence entre compiler et téléverser un programme ?

Ce sont les deux étapes pour envoyer du code dans la carte :

- **Compiler (Vérifier)** : L'ordinateur vérifie s'il y a des fautes de syntaxe dans votre code, puis le traduit en code binaire (le langage machine que comprend le microcontrôleur). Cela fonctionne même si la carte n'est pas branchée.
- **Téléverser (Uploader)** : L'ordinateur compile d'abord le programme, puis utilise la liaison USB pour copier ce binaire dans la mémoire flash de la carte Arduino. La carte redémarre et commence immédiatement à exécuter le code.

Que signifie le terme « baud rate » dans une communication série ?

C'est la vitesse de transmission des données sur la liaison série :

- Elle s'exprime en **bits par seconde**.
- Par exemple, une vitesse de **9600** bauds signifie que la carte peut envoyer environ 9600 bits d'information par seconde.
- D'autres vitesses plus rapides sont courantes, comme **115200** bauds.
- Pour que la communication fonctionne, l'expéditeur et le destinataire doivent absolument être réglés sur la même vitesse.

Pourquoi la valeur configurée dans `Serial.begin()` doit-elle correspondre à celle du moniteur série ?

C'est une question de synchronisation temporelle :

- La liaison série n'a pas de fil d'horloge partagé. L'ordinateur et l'Arduino doivent donc "deviner" le début et la fin de chaque bit en comptant le temps.
- Si l'Arduino envoie des données à 9600 bauds mais que le moniteur série écoute à 115200 bauds, l'ordinateur va lire le signal beaucoup trop vite.
- **Le résultat** : Les données reçues n'ont plus aucun sens et le moniteur affiche des caractères incompréhensibles (des rectangles, des points d'interrogation ou du texte bizarre).

Comment tester un capteur avant de l'intégrer dans un projet complet ?

Pour éviter de chercher des pannes compliquées, testez toujours vos composants un par un :

1. **Câblage minimal** : Branchez uniquement ce capteur sur votre plaque d'essai (breadboard).
2. **Code de test simple** : Utilisez un exemple de base fourni par la bibliothèque du capteur (menu Fichier > Exemples).
3. **Affichage direct** : Envoyez les mesures brutes sur le moniteur série.
4. **Test physique** : Soufflez sur le capteur de température, ou passez votre main devant le capteur de distance pour voir si les valeurs changent en direct.

Comment savoir si une erreur vient du code, du câblage ou du capteur ?

Procédez par élimination logique :

- **Erreur de code** : Le moniteur série affiche des valeurs, mais le programme ne réagit pas comme prévu (ex: le buzzer ne sonne pas). Revérifiez vos conditions `if` et vos variables.
- **Erreur de câblage** : Le capteur renvoie toujours la même valeur aberrante (ex: -999 ou 0) ou chauffe. Vérifiez avec vos yeux chaque fil et utilisez le multimètre.
- **Capteur en panne** : Si le câblage est parfait et que le code de test officiel ne fonctionne pas, essayez un autre capteur identique si vous en avez un.

Quels composants du kit peuvent être utilisés pour réaliser une station intelligente de surveillance d'une salle ?

Pour concevoir ce projet pratique, on combine ces modules du kit :

- **Pour capter l'état de la salle** : Le capteur **DHT11** (mesure température/humidité), une photorésistance **LDR** (pour savoir si les lumières sont allumées) et un capteur **PIR** (détection de mouvements suspects).
- **Pour alerter et afficher** : Une **LED RVB** (qui passe au rouge en cas de problème), un **Buzzer** (alarme sonore) et un **écran LCD** pour afficher les mesures.
- **Pour agir** : Un **relais** pour allumer une vraie lampe de la salle.

Classification et rôle des composants du kit

Voici comment regrouper les éléments du matériel distribué :

- **Le Cerveau** : La carte Arduino Uno.
- **Le Prototypage** : La breadboard (plaque d'essai) et les fils de connexion.
- **Les Capteurs** : DHT11 (climat), LDR (lumière), HC-SR04 (distance), PIR (mouvement), Potentiomètre (bouton rotatif).
- **Les Actionneurs** : LEDs de couleurs, LED RVB, Buzzer (son), Servomoteur SG90 (mouvement angulaire), Relais (interrupteur électrique).

Conclusion — Points clés à retenir

- **Simplicité** : Arduino est parfait pour démarrer et comprendre les bases de l'électronique de façon intuitive.
- **Évolution connectée** : L'ESP32 prend naturellement le relais dès qu'on a besoin d'objets connectés (IoT) ou de performances élevées.
- **Bonnes pratiques** : Testez toujours vos capteurs séparément avec des programmes simples pour ne pas vous perdre dans les pannes complexes.
- **Méthodologie** : Quand un montage ne marche pas, gardez votre calme et vérifiez d'abord le code, ensuite le câblage, et enfin le matériel.

Merci pour votre attention. Place maintenant à la démonstration pratique !